

Class XII | Unit 2 | weight 15%

Pointers

Board weight about 15% | 5 theory periods and 5 lab periods

A pointer holds the address of a variable. With that address the program can read or change the variable itself. Pointers are short to write and easy to lose marks on, so the operators and the types have to be exact.

What you should be able to do

- Explain why pointers are used and what a memory address is.
- Use the address-of operator and the dereference operator.
- Declare and initialise a pointer.
- Write a program that displays an address and a value, and a function that changes variables through pointers.

Path through the lecture

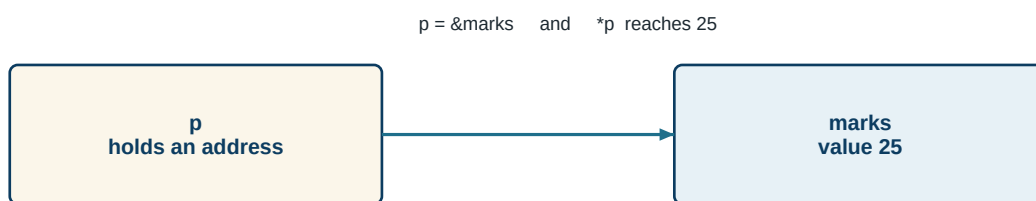
- 1 Draw a variable as a box and a pointer as a box that contains the address of the first box.
- 2 Type the single-variable program and read both the address and the value.
- 3 Pass two addresses into a function and show that the originals change.

2.1 Addresses and pointer variables

Pointer. A variable whose value is the address of another variable.

Every variable occupies memory, and that place has an address. The **address-of operator** & produces the address of a variable. The **dereference operator** * uses an address to reach the variable that lives there. Declaring `int *p;` means p is a pointer to an int. It does not mean that p is an int.

Initialising the pointer matters. `int *p = &marks;` stores the address of marks in p. After that, *p and marks name the same value. Changing *p changes marks. A pointer that has never been set must not be dereferenced. It does not yet point at a variable you own.



The pointer stores where the variable lives, not a second copy of the value.

Figure 2.1 p stores the address of marks. *p is the value 25.

Pointers matter because a function can receive an address and change the caller's variable, because dynamic structures such as linked data are built from addresses, and because a large value can be shared by passing its address rather than copying it. In this unit the required skill is the simple use: take an address, store it, and reach the value.

Program 2.1 Address and value through a pointer

```
#include <iostream>
using namespace std;

int main() {
    int marks = 25;
    int *p = &marks;
    cout << "Value = " << marks << endl;
    cout << "Address stored in p = " << p << endl;
    cout << "Through pointer = " << *p << endl;
    *p = 40;
    cout << "New value = " << marks << endl;
    return 0;
}
```

The address printed for p changes from run to run. That is expected. The value 25, and the new value 40 after *p = 40, stay the same. The program shows both operators: &marks goes into p, and *p comes back to the value.

2.2 Pointers as function arguments

A function parameter of type `int *` receives an address. Inside the function, *a reaches the original variable. At the call, pass &first, not first. Passing first would pass the number, and the types would not match.

Program 2.2 Double two integers through pointers

```
#include <iostream>
using namespace std;

void doubleBoth(int *a, int *b) {
    *a = *a * 2;
    *b = *b * 2;
}

int main() {
    int first, second;
    cout << "Enter two integers: ";
    cin >> first >> second;
    doubleBoth(&first, &second);
    cout << "First = " << first << endl;
    cout << "Second = " << second << endl;
    return 0;
}
```

Input 4 and 7 prints 8 and 14. This is the pointer form of pass by reference. The & form you used in Class XI (`int &n`) and the pointer form (`int *n` plus `*n`) both let a function change the original. The pointer form makes the address visible, which is the point of this unit.

Exam point

In an exam, the declaration `int *p`; the assignment `p = &x`; and the use `*p` are three separate marks. Write all three. State the type the pointer points to.

Key terms

Term	Meaning to write in an answer
Address	The location of a variable in memory.
Pointer	A variable that stores an address.
&	The address-of operator. &x is the address of x.
*	The dereference operator. *p is the object p points to.

Term	Meaning to write in an answer
NULL	A pointer value that points nowhere. Do not dereference it.

Bring it together

- A pointer stores an address. & takes an address. * follows an address.
- The declaration `int *p` says that *p is an int, so p points to an int.
- A function changes a caller's variable when it receives the variable's address and assigns through *.

Do this in the practical

- 1 Display the address and the value of a variable using a pointer.
- 2 Pass the addresses of two variables to a function and double both values.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. The operator & produces:

- A. The address of its operand
- B. The square of a number
- C. A new class
- D. A file mode

2. If p points to marks, then *p is:

- A. The value of marks
- B. The name of the file
- C. Always zero
- D. The layer number

3. `int *p;` declares:

- A. A pointer to int
- B. An int that cannot change
- C. A string
- D. A table

4. Which call matches the parameter `int *a`?

- A. `doubleBoth(&first, &second)`
- B. `doubleBoth(first, second)`
- C. `doubleBoth(*first, *second)`
- D. `doubleBoth(a, b, c)`

5. Assigning `*p = 40;` when p points to marks:

- A. Changes marks to 40
- B. Deletes the program
- C. Changes the topology
- D. Closes a file

6. A pointer should be initialised so that:

- A. It holds the address of a real variable before it is dereferenced
- B. It holds a class name
- C. It replaces main
- D. It stores a flowchart

7. Passing a pointer into a function is useful because:

- A. The function can change the original variable
- B. The keyboard becomes wireless
- C. ROM becomes volatile
- D. Arrays become illegal

8. The value printed as the address of a local variable:

- A. Can differ from one run to the next
- B. Is always 25
- C. Is the source code
- D. Is the SRS

Answer key

Q	Answer	Why this is the answer
1	A	& is the address-of operator.
2	A	Dereferencing p reaches the variable it points to.
3	A	The * in the declaration belongs with p and makes p a pointer.
4	A	The function expects addresses, and &first is an address.
5	A	*p names the same memory as marks.
6	A	Dereferencing an unset pointer does not refer to a known variable.
7	A	The function receives the address and writes through it.
8	A	The system chooses where to place the variable each run.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Define a pointer and state the purpose of & and *.

Model answer

A pointer is a variable that stores the address of another variable. The operator & gives the address of a variable. The operator * uses a pointer to reach the variable at that address. After `int *p = &marks`, *p and marks refer to the same value.

Q2. Write the declaration and initialisation of a pointer to an int variable named marks.

Model answer

`int marks = 25; int *p = &marks;` The declaration `int *p` says p points to an int. The initialiser stores the address of marks. *p then reads or changes marks.

Q3. Why is a pointer parameter able to change the caller's variable?

Model answer

The caller passes the address, for example &first. The parameter holds that address. An assignment to *a writes into the caller's variable, not into a copy of the number.

Q4. Give two uses of pointers.

Model answer

A function can modify the variables of its caller by receiving their addresses. Programs also use addresses to build data structures whose size can grow, and to share a large object without copying it.

Q5. What is wrong with dereferencing a pointer that has not been initialised?

Model answer

The pointer does not yet hold the address of a variable the program owns. Using * on it reaches an unknown location. Initialise the pointer with &variable, or set it only from a known object, before you dereference it.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain pointers with a program that shows the address and the value of a variable.

- 1 Define pointer, address, & and *.
- 2 Declare an int and a pointer, and initialise the pointer with the address of the int.
- 3 Print the variable, the pointer, and the dereferenced pointer.
- 4 Change the variable through *p and print the variable again to show it changed.
- 5 State one reason this is useful in a function call.

Question 2. Write a function that receives the addresses of two integers and doubles both. Explain each operator at the call and inside the function.

- 1 Write the function heading with two int* parameters.
- 2 Inside, assign *a = *a * 2 and the same for b.
- 3 In main, read two integers and call the function with &first and &second.
- 4 Explain why the arguments need & and why the body needs *.
- 5 Give the sample 4 and 7 becoming 8 and 14.